Image Upscaling Using a Convolutional Autoencoder

R. Kumar*
*Department of Mechanical Engineering, University of Tennessee at Chattanooga
Chattanooga, TN USA

I. Introduction

Image super-resolution is the problem of recovering a high-resolution image from a degraded low-resolution input. The task arises in medical imaging, satellite photography, and video surveillance, where captured images are routinely too coarse for direct analysis [1]. Classical approaches such as bicubic interpolation produce blurry results because they contain no learned prior about the appearance of natural images. Data-driven methods instead learn this prior from large collections of paired training examples, allowing the reconstruction to fill in plausible high-frequency detail.

This work implements a convolutional autoencoder [2] trained to upscale photographs of cats and dogs from 28×28 pixels to 56×56 pixels—a factor of two in each spatial dimension. The dataset contains 25,000 training images and 12,500 test images drawn from a mixed cat and dog collection. Since the task is purely reconstructive, class labels are discarded entirely; the model is trained in an unsupervised fashion by minimising the pixel-level mean squared error between its output and a reference 56×56 resize of each original image.

Two variants of the same architecture are evaluated. The first operates on single-channel grayscale inputs, reducing the reconstruction problem to spatial structure alone. The second processes three-channel RGB inputs, requiring the network to jointly recover both spatial detail and chromatic information. Because the architecture is identical across variants—only the first and last convolutional layers change to match the channel count—the experimental setup provides a controlled comparison of the trade-off between colour fidelity and spatial sharpness at fixed model capacity.

The remainder of this report is organised as follows. Section II reviews convolutional network theory and autoencoder fundamentals. Section III details the architecture, loss function, and training procedure. Section IV presents the complete layer-by-layer configuration. Section V analyses the experimental results for both variants, and Section VI summarises the findings and suggests directions for improvement.

II. Background

A. Why Convolutional Networks for Image Data

A fully-connected (MLP) layer treats every input pixel as an independent feature with its own weight. This is computationally expensive and ignores two structural properties that images universally possess. The first is *locality*: nearby pixels are more strongly correlated than distant ones, as they tend to belong to the same edge, texture region, or object boundary. The second is *location independence*: a meaningful pattern—a cat ear, a fur texture—looks the same regardless of where it appears in the image. A fully-connected layer must learn a separate detector for each position, which wastes parameters and does not generalise across translations.

Convolutional layers address both issues by applying a small filter (kernel) of size $k \times k$ to every spatial position in the input. Because the same filter is used everywhere, the layer is *equivariant* to translation: if a feature shifts in the input, its activation shifts by the same amount in the feature map. This weight sharing drastically reduces parameter count and acts as an implicit spatial regulariser [3]. For colour images, each filter extends across all input channels and is therefore a third-order tensor of shape $k \times k \times C_{\text{in}}$; applying C_{out} such filters produces an output with C_{out} channels.

B. The Convolution Operation and Output Dimensions

For an input feature map of spatial height h_{in}, a kernel of size k, symmetric zero-padding p, and stride s, the output height is

$$h_{\text{out}} = \left\lfloor \frac{h_{\text{in}} + 2p - k}{s} \right\rfloor + 1\,. \tag{1}$$

All convolutional layers in this work use $k = 3$, $p = 1$, $s = 1$, which by (1) yields $h_{\text{out}} = h_{\text{in}}$: spatial dimensions are preserved within each convolutional block. Spatial change is introduced only by max-pooling (in the encoder) and bilinear upsampling (in the decoder), both described below.

C. Max Pooling and Spatial Compression

Max pooling with a 2×2 window and stride 2 replaces each non-overlapping 2×2 neighbourhood of activations with its maximum:

$$z_{i,j} = \max_{(m,n) \in \mathcal{N}_{i,j}} a_{m,n} \tag{2}$$

where $\mathcal{N}_{i,j}$ denotes the 2×2 window at position (i, j). This halves both spatial dimensions ($h_{\text{out}} = h_{\text{in}}/2$). In an encoder, successive pooling steps serve two roles: they increase the effective receptive field of subsequent filters, and they force the network to develop progressively more abstract, spatially compressed representations—a prerequisite for learning a meaningful bottleneck code.

D. Autoencoders: Structure and Theory

An autoencoder is an unsupervised neural network trained to reconstruct its own input [2]. It belongs to the class of *generative* models that attempt to characterise the data distribution $p(\mathbf{x})$, in contrast to *discriminative* models that estimate a conditional $p(y \mid \mathbf{x})$ for a specific label variable.

No class annotations are needed: the training signal comes entirely from comparing the output to the input.

The architecture is split into two sub-networks. The *encoder* e maps an input $\mathbf{x}$ to a lower-dimensional *code* $\mathbf{h}$ (also called the *latent representation* or *bottleneck*):

$$\mathbf{h} = e(\mathbf{x}) \,. \tag{3}$$

The *decoder* d maps $\mathbf{h}$ back to a reconstruction $\hat{\mathbf{x}}$:

$$\hat{\mathbf{x}} = d(\mathbf{h}) = d(e(\mathbf{x})) \,. \tag{4}$$

Training minimises the reconstruction loss over a dataset of N examples. For pixel-level regression, the standard choice is the mean squared error:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \left\| \mathbf{x}^{(n)} - d\Big(e(\mathbf{x}^{(n)})\Big) \right\|^2 \,. \tag{5}$$

Three properties characterise autoencoders in practice. First, they are *data-specific*: a model trained on cat and dog photographs will not generalise meaningfully to satellite imagery, because the encoder is tuned to the manifold structure of its training distribution. Second, the reconstruction is *lossy*: the bottleneck code is smaller than the input, so the decoder must infer missing detail from learned statistical regularities. Third, the model is *self-supervised*: the only supervision required is the input itself, making large unlabelled datasets directly usable.

E. Autoencoders Versus Linear Dimensionality Reduction

Principal Component Analysis (PCA) solves the same reconstruction objective (5) under the constraint that the encoder and decoder are each a single linear transformation with tied weights. The solution is a set of orthogonal directions of maximum variance. A neural autoencoder lifts the linearity constraint: deeper encoders with nonlinear activations can represent curved manifolds and cross-channel correlations that PCA cannot capture. When the encoder and decoder consist of convolutional layers, the model additionally exploits the spatial structure of image data—the same argument that motivates convolutional networks over fully-connected ones—making convolutional autoencoders particularly effective for image super-resolution, denoising, and colourisation tasks.

III. Methodology

A. Dataset and Preprocessing

The dataset consists of 25,000 JPEG training images and 12,500 test images drawn from the Kaggle Dogs vs. Cats collection. Each image is resized to two resolutions using PIL's Lanczos resampler: 28×28 (the network input) and 56×56 (the reconstruction target). Pixel values are normalised to $[0, 1]$ by dividing by 255. The resulting arrays are cached as `.npy` files on the first run; subsequent runs load directly from disk, reducing data preparation time from several minutes to under one second for the full training set.

The grayscale and RGB pipelines share the same structure but differ at the loading and tensor-shaping steps:

- **Grayscale**: Images are loaded with `PIL.Image.convert('L')`, which collapses all three colour channels into a single luminance channel. The resulting arrays are reshaped to $(N, 1, H, W)$.
- **RGB**: Images are loaded with `convert('RGB')`, producing arrays of shape $(N, H, W, 3)$ in height-width-channel order. `numpy.transpose(0,3,1,2)` reorders the axes to $(N, 3, H, W)$ to match PyTorch's channel-first (NCHW) convention.

Separate cache files (`*_rgb.npy`) prevent the two preprocessing pipelines from overwriting each other.

B. Encoder

The encoder maps an input of shape $(N, C, 28, 28)$—where $C = 1$ for grayscale and $C = 3$ for RGB—to a bottleneck tensor of shape $(N, 128, 7, 7)$ through two max-pooling stages. Each pooling step halves both spatial dimensions: $28 \to 14 \to 7$. The convolutional blocks between pooling layers all use 3×3 kernels with same-padding, so by (1) spatial dimensions are preserved within each block. Filter counts increase as resolution decreases (from 32 at 28×28 to 128 at 7×7), keeping the total representational capacity roughly constant across depth. A `Dropout2d` layer with $p = 0.25$ is applied at the bottleneck, which randomly zeroes entire feature channels during training and regularises the compressed representation.

C. Decoder

The decoder inverts the encoder by replacing max-pooling with bilinear upsampling. Three successive $\times 2$ upsamplings expand the spatial dimensions $7 \to 14 \to 28 \to 56$. The key asymmetry relative to the encoder is deliberate: the encoder performs *two* pooling steps while the decoder performs *three* upsampling steps. The extra stage is the architectural mechanism that produces the $2\times$ super-resolution output: the decoder restores the original 28×28 spatial extent and then goes one level further to reach 56×56.

Bilinear upsampling was chosen over transposed convolutions because the latter are prone to checkerboard artifacts caused by uneven kernel overlap [4]. Each upsampling operation is followed by a trainable convolution, which allows the network to refine the interpolated activations rather than relying on the fixed interpolation pattern alone.

D. Loss Function

The network is trained to minimise the mean squared error between the predicted image $\hat{\mathbf{y}}$ and the 56×56 ground-truth resize $\mathbf{y}$:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N \cdot C \cdot H \cdot W} \sum_{n,c,h,w} \left(\hat{y}_{n,c,h,w} - y_{n,c,h,w}\right)^2 \,. \tag{6}$$

Here $C = 1$ for grayscale and $C = 3$ for RGB; PyTorch's `MSELoss` averages over all tensor elements, making the scalar loss value roughly comparable across the two variants. MSE is

the standard choice for super-resolution because it is differentiable everywhere and penalises large pixel errors more heavily than small ones [1]. A known limitation is that MSE-optimal predictions tend to be slightly blurry: because the loss rewards the per-pixel mean of all plausible high-resolution images, the network learns to produce smooth, safe predictions rather than sharp but uncertain ones.

E. Batch Normalisation and Regularisation

Batch normalisation is applied after every convolutional layer. For a mini-batch of activations $\{x_i\}_{i=1}^{m}$, the batch statistics $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are used to normalise each activation:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}\,, \qquad y_i = \gamma \hat{x}_i + \beta\,, \tag{7}$$

where γ and β are learnable scale and shift parameters that restore the representational capacity of the layer after normalisation. By keeping activation distributions centred and unit-scale throughout training, batch normalisation allows the use of higher learning rates and substantially reduces the sensitivity to weight initialisation [5].

F. Optimiser and Learning Rate Schedule

The Adam optimiser [6] maintains exponentially decaying estimates of the gradient's first moment m_t and second moment v_t:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\,\nabla\mathcal{L}_t \tag{8}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\,(\nabla\mathcal{L}_t)^2 \tag{9}$$

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\,\hat{m}_t \tag{10}$$

with bias-corrected estimates $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$, and default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$. The per-parameter adaptive learning rates in Adam handle the varying gradient magnitudes across encoder and decoder layers naturally. A cosine annealing schedule gradually reduces η to near zero over the full training run, helping the model converge to a sharper minimum during the final epochs rather than oscillating around it.

IV. Model Configuration

Figure 1 gives a visual overview of the proposed convolutional autoencoder, and Table I lists the exact layer configuration. All convolutional layers use 3×3 kernels with same-padding and are followed by batch normalisation and ReLU unless otherwise noted. The sole differences between the grayscale ($C = 1$) and RGB ($C = 3$) variants are the first encoder convolution and the final decoder convolution, both marked †.

The architecture follows a symmetric encoder–decoder pattern in channel flow, but it is intentionally asymmetric in spatial scaling. On the encoder side, two convolutional blocks at full resolution extract low-level edge and texture cues before max-pooling reduces the feature maps to 14×14 and then 7×7. As the spatial size shrinks, the channel depth increases from 32 to 64 and then to 128, allowing the network to replace raw pixel detail with a denser learned feature representation.

At the bottleneck, the tensor $(128, 7, 7)$ acts as a compact latent code that retains semantic structure while discarding fine-grained redundancy. A `Dropout2d` layer is applied at this stage so that the decoder does not over-rely on any single feature channel. This regularisation is especially useful for natural image reconstruction, where multiple channels can otherwise specialise too narrowly to particular fur textures, edges, or background patterns.

The decoder reverses the abstraction process in a reconstruction-oriented manner. After an initial $128 \rightarrow 64$ convolution, the feature maps are progressively enlarged using bilinear upsampling, with intermediate convolutions reducing the channel depth from 64 to 32 to 16 while refining the interpolated activations. The final convolution maps these 16 channels back to C output channels, and a sigmoid activation constrains each pixel to the normalised intensity range $[0, 1]$. This sequence lets the model first rebuild coarse global structure and then recover finer local detail at the higher spatial resolution.

The following subsections discuss each principal design choice.

Filter progression (32 / 64 / 128). Natural photographs contain far richer texture and edge detail than simple digit images. Using more filters per layer—relative to a basic autoencoder tutorial—gives the encoder sufficient capacity to represent fur, shadow gradients, and background clutter. Increasing filter counts as spatial resolution decreases keeps the total information content roughly constant through the encoder depth.

Three upsampling stages vs. two pooling stages. The asymmetry is the core architectural choice that enables super-resolution. Two max-pooling operations compress the spatial extent $28 \rightarrow 14 \rightarrow 7$, while three bilinear upsamplings expand it $7 \rightarrow 14 \rightarrow 28 \rightarrow 56$. The net effect is a $2\times$ increase in spatial resolution relative to the input.

Bilinear upsampling over transposed convolution. Transposed convolutions can learn upsampling weights from data but are susceptible to checkerboard patterns when the kernel size is not a multiple of the stride [4]. Bilinear upsampling is deterministic and smooth; the subsequent trainable convolution still allows the network to refine the interpolated activations.

Batch size 128 and 30 epochs. Batches of 128 images consistently mix cat and dog examples, giving stable gradient estimates. The training loss was observed to plateau around epoch 20–25; training for 30 epochs ensures convergence while the best checkpoint is retained throughout.

V. Results and Discussion

A. Training Behaviour

Both models were trained with the Adam optimiser under a cosine annealing schedule for 30 epochs with a batch size of 128. The training loss decreased monotonically across all epochs for both variants, confirming that the architecture and the optimisation setup are well-conditioned. The grayscale model converged to a lower final MSE than the RGB model,

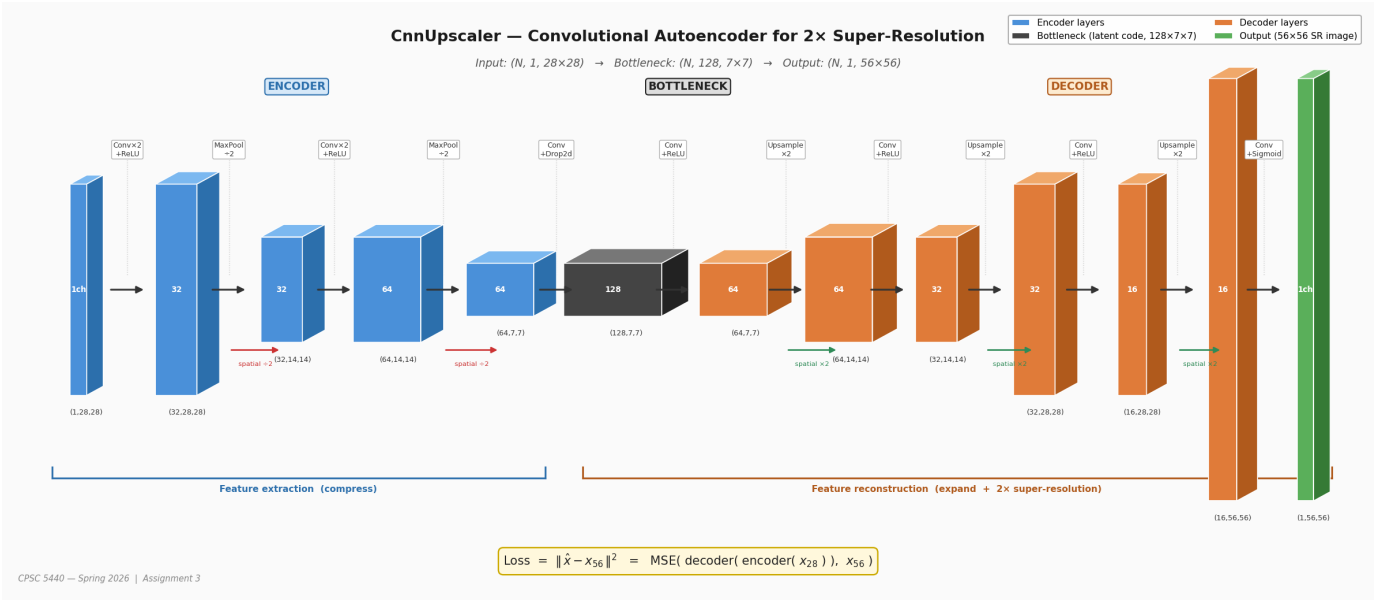

Fig. 1. Visual overview of the convolutional autoencoder used for $2\times$ super-resolution. The encoder extracts local image features from the 28×28 input through stacked convolutional blocks and two max-pooling operations, compressing the representation to a bottleneck of size $(128, 7, 7)$. The decoder then reconstructs and enlarges the representation through convolutional refinement and three bilinear upsampling stages, producing a 56×56 output image. The additional decoder upsampling stage relative to the encoder pooling depth is the key design choice that converts a standard autoencoder into a super-resolution model.

which is consistent with the intuition that minimising reconstruction error over a single intensity channel is simpler than jointly optimising a three-channel objective whose components are correlated. The best checkpoint based on training MSE was saved automatically after each epoch improvement.

B. Grayscale Results

Figure 2 shows predictions from the grayscale model on eight randomly selected test images. Each column displays the 28×28 low-resolution input (top row), the 56×56 network output (middle row), and the 56×56 ground-truth (bottom row).

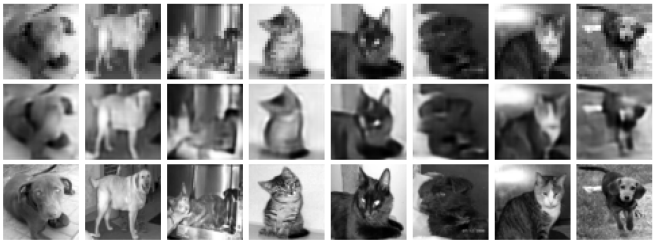

Fig. 2. Grayscale model predictions on eight randomly selected test images. *Top*: 28×28 low-resolution inputs. *Middle*: 56×56 reconstructions produced by the convolutional autoencoder. *Bottom*: 56×56 ground-truth obtained by directly downsampling the original image. Broad spatial structure and outline are recovered well; fine textures such as whiskers and eye boundaries remain somewhat softened, a characteristic consequence of MSE-based training.

The reconstructed images recover the broad spatial structure of each photograph well: body outline, fur regions, and background contrast are all clearly present. Compared to the low-resolution input, the outputs are noticeably sharper. The primary limitation appears in high-frequency detail—whiskers, eye boundaries, and individual hair strands tend to be softened. This blurring is a well-documented consequence of MSE-based training [1]: because the loss penalises large pixel deviations, the network is incentivised to produce a smooth per-pixel average rather than commit to sharp but uncertain fine structure.

C. Colour (RGB) Results

Figure 3 shows predictions from the RGB model evaluated on the same set of test images.

Fig. 3. RGB model predictions on eight randomly selected test images. *Top*: 28×28 colour inputs. *Middle*: 56×56 colour outputs produced by the convolutional autoencoder. *Bottom*: 56×56 ground-truth. Coat colour, background tone, and gross colour gradients are recovered accurately; spatial sharpness is visibly lower than the grayscale variant at equal model capacity, reflecting the increased complexity of a three-channel reconstruction objective.

The RGB model correctly recovers colour content across test samples: coat colour, background tone, and gross colour gradients are all approximately right. However, spatial sharpness is visibly lower than in the grayscale case for the same architecture and training budget. Two contributing factors

TABLE I

LAYER-BY-LAYER ARCHITECTURE OF THE CONVOLUTIONAL AUTOENCODER. LAYERS MARKED † DIFFER BETWEEN THE GRAYSCALE ($C=1$) AND RGB ($C=3$) VARIANTS; ALL OTHER LAYERS ARE SHARED. *BN* = BATCH NORMALISATION APPLIED AFTER EACH CONVOLUTION. SPATIAL DIMENSIONS ARE SHOWN AS HEIGHT × WIDTH; BATCH AXIS N IS OMITTED FOR BREVITY.

Stage	**Layer**	**Operation**	**Grayscale output shape**	**RGB output shape**	**Notes**
Input	—	—	$(1, 28, 28)$	$(3, 28, 28)$	†
Encoder	Conv(C→32) + BN + ReLU	3×3, $p=1$	$(32, 28, 28)$	$(32, 28, 28)$	† first layer
	Conv(32→32) + BN + ReLU	3×3, $p=1$	$(32, 28, 28)$	$(32, 28, 28)$	
	MaxPool(2)	2×2, $s=2$	$(32, 14, 14)$	$(32, 14, 14)$	$28 \to 14$
	Conv(32→64) + BN + ReLU	3×3, $p=1$	$(64, 14, 14)$	$(64, 14, 14)$	
	Conv(64→64) + BN + ReLU	3×3, $p=1$	$(64, 14, 14)$	$(64, 14, 14)$	
	MaxPool(2)	2×2, $s=2$	$(64, 7, 7)$	$(64, 7, 7)$	$14 \to 7$
	Conv(64→128) + BN + ReLU	3×3, $p=1$	$(128, 7, 7)$	$(128, 7, 7)$	Bottleneck
	Dropout2d($p=0.25$)	—	$(128, 7, 7)$	$(128, 7, 7)$	
Decoder	Conv(128→64) + BN + ReLU	3×3, $p=1$	$(64, 7, 7)$	$(64, 7, 7)$	
	Upsample($\times 2$)	Bilinear	$(64, 14, 14)$	$(64, 14, 14)$	$7 \to 14$
	Conv(64→32) + BN + ReLU	3×3, $p=1$	$(32, 14, 14)$	$(32, 14, 14)$	
	Upsample($\times 2$)	Bilinear	$(32, 28, 28)$	$(32, 28, 28)$	$14 \to 28$
	Conv(32→16) + BN + ReLU	3×3, $p=1$	$(16, 28, 28)$	$(16, 28, 28)$	
	Upsample($\times 2$)	Bilinear	$(16, 56, 56)$	$(16, 56, 56)$	$28 \to 56$
	Conv(16→C) + Sigmoid	3×3, $p=1$	$(1, 56, 56)$	$(3, 56, 56)$	† output

stand out. First, the optimisation problem is more complex: the model must simultaneously minimise reconstruction error across three correlated channels, and the resulting loss landscape is harder to navigate with the same parameter count. Second, because the MSE (6) averages over all three channels equally, a large localised error in one channel (for instance, a wrong hue region) exerts a gradient pressure primarily on that channel, leaving the spatial structure of the other two channels less constrained. This allows colour inaccuracies to persist without strongly pulling the spatial predictions toward sharper boundaries.

D. Grayscale vs. Colour Comparison

Table II summarises the principal qualitative differences observed between the two variants.

TABLE II

QUALITATIVE COMPARISON OF GRAYSCALE AND RGB SUPER-RESOLUTION OUTPUTS EVALUATED ON THE HELD-OUT TEST SET. HIGHER SPATIAL SHARPNESS AND LOWER TRAINING MSE ARE DESIRABLE; COLOUR INFORMATION IS AVAILABLE ONLY IN THE RGB VARIANT.

Aspect	**Grayscale**	**RGB**
Spatial sharpness	Higher	Slightly lower
Colour information	None	Approximately correct
Training MSE	Lower	Higher
Optimisation ease	Simpler	More complex
Output channels	1	3

The grayscale model produces sharper spatial reconstructions at lower MSE, corroborating the assignment's note that simpler autoencoders benefit from grayscale inputs. The RGB model trades some spatial precision for colour fidelity, which matters for perceptual quality even when the numerical error is slightly higher. For applications where colour is essential—photo restoration, medical imaging with colour staining, or enhancing coloured satellite imagery—the RGB variant is the appropriate choice. For tasks where spatial resolution is the primary concern—detection, segmentation, or anomaly localisation—the grayscale model is preferable at equal model capacity.

VI. CONCLUSION

Two convolutional autoencoders were implemented in PyTorch [7] and trained to perform $2\times$ image super-resolution on a mixed cat and dog dataset: one on single-channel grayscale inputs and one on three-channel RGB inputs. Both share an identical encoder–decoder structure. The encoder applies two max-pooling stages to compress 28×28 inputs to a 7×7 bottleneck of 128 channels; the decoder applies three bilinear upsampling stages to expand this back to 56×56. The asymmetry—one more upsampling step than pooling steps—is the deliberate architectural decision that produces the factor-of-two super-resolution output. Batch normalisation and spatial dropout regularise training, and Adam with cosine annealing provides stable convergence over 30 epochs.

The grayscale model converged to a lower training MSE and produced sharper spatial reconstructions, consistent with the theoretical expectation that a single-channel target is an easier optimisation objective. The RGB model recovered colour content well but at the cost of slightly softer spatial detail, reflecting the additional complexity of a three-channel loss landscape at fixed model capacity.

Both results confirm that MSE-trained autoencoders learn mean-like, slightly blurred reconstructions rather than sharp posterior modes. The natural extension of this work is to replace or augment the MSE objective with a perceptual loss

computed in an intermediate feature space, or to add an adversarial discriminator that explicitly rewards sharp, realistic outputs. Either modification could improve fine-detail recovery without requiring any change to the underlying encoder–decoder architecture.

Appendix A

The Google Colab notebook for this assignment is attached below:

- CNNUpscalingRGB
- CNNUpscalingGray

The GitHub repository for the project can be found at:

- github

The YouTube video for this assignment can be found at:

- YouTube

References

1 Dong, C., Loy, C. C., He, K., and Tang, X., "Image super-resolution using deep convolutional networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 2, pp. 295–307, 2016.

2 Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*. Cambridge, MA: MIT Press, 2016.

3 LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

4 Odena, A., Dumoulin, V., and Olah, C., "Deconvolution and checkerboard artifacts," in *Distill*, 2016.

5 Ioffe, S. and Szegedy, C., "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

6 Kingma, D. P. and Ba, J., "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, 2015.

7 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, vol. 32.